Q1. What is the difference between a parametric and a non-parametric model?

Answer: A parametric model has a fixed number of parameters regardless of the amount of training data, such as linear regression or logistic regression. A non-parametric model grows in complexity with more data and does not assume a fixed form, such as k-nearest neighbors or kernel density estimation. Parametric models are faster but make stronger assumptions, while non-parametric models are more flexible but require more data and computation.

Q2. What is gradient descent and what are its main variants?

Answer: Gradient descent is an optimization algorithm that iteratively updates model parameters in the direction that reduces the loss function. Batch gradient descent uses the entire dataset per update, stochastic gradient descent uses one sample per update, and mini-batch gradient descent uses a small subset per update. Mini-batch is the most commonly used in practice as it balances speed and stability.

Q3. What is XGBoost and why is it popular?

Answer: XGBoost is an optimized gradient boosting algorithm that builds decision trees sequentially, each one correcting the errors of the previous. It is popular because it is fast, handles missing values automatically, includes built-in regularization to reduce overfitting, and consistently produces strong results on structured tabular data. It has been widely used in machine learning competitions and real-world industry applications.

Q4. What is the curse of dimensionality?

Answer: The curse of dimensionality refers to the problems that arise when the number of input features is very large. As dimensions increase, data becomes increasingly sparse, distances between points become less meaningful, and exponentially more training data is needed to cover the feature space. This causes models to overfit and makes many algorithms slow and ineffective without dimensionality reduction or feature selection.

Q5. What is dimensionality reduction and what is PCA?

Answer: Dimensionality reduction is the process of reducing the number of input features while keeping as much useful information as possible. Principal Component Analysis (PCA) is the most common technique, which transforms the original features into a smaller set of uncorrelated components called principal components that capture the maximum variance in the data. It helps reduce overfitting, speed up training, and remove noise.

Q6. What is the difference between PCA and feature selection?

Answer: PCA creates new features by combining the original features into principal components, which are transformations of the original variables. Feature selection simply picks the most relevant original features and removes the rest without creating new ones. PCA is useful when features are correlated and you want to reduce noise, while feature selection is preferred when interpretability of the original features is important.

Q7. What is the difference between online learning and batch learning?

Answer: Batch learning trains the model on the entire dataset at once and then deploys it without further updates until retrained. Online learning updates the model incrementally as each new data point arrives, allowing it to adapt to new information continuously. Online learning is useful when data arrives as a stream or when the data distribution changes over time, while batch learning is simpler and works well when the full dataset is available upfront.

Q8. What is concept drift?

Answer: Concept drift occurs when the statistical relationship between input features and the target variable changes over time, causing a model trained on old data to become less accurate on new data. For example, a fraud detection model may become outdated as fraudsters change their behavior. It is handled by monitoring model performance over time and retraining on recent data or using online learning algorithms that adapt continuously.

Q9. What is semi-supervised learning?

Answer: Semi-supervised learning trains a model using a small amount of labeled data combined with a large amount of unlabeled data. It is useful when labeling data is expensive or time-consuming but collecting raw data is easy. The model uses the structure of unlabeled data, such as natural clusters or patterns, to improve its performance beyond what would be possible with only the small labeled set.

Q10. What is active learning?

Answer: Active learning is a strategy where the model selects the most informative examples from unlabeled data and asks a human expert to label only those examples. Instead of labeling all data randomly, this focuses labeling effort on the samples where the model is most uncertain or where labels would most improve performance. This achieves good accuracy with significantly fewer labeled examples, reducing the cost of data annotation.

Q11. What is multi-task learning?

Answer: Multi-task learning trains a single model on multiple related tasks at the same time, sharing learned representations across all tasks. By learning tasks together, the model benefits from information shared between them, which improves generalization and reduces the need for large labeled datasets for each task individually. It also acts as a regularizer, since the model cannot overfit to noise specific to any single task.

Q12. What is transfer learning?

Answer: Transfer learning takes a model trained on one task or dataset and reuses its learned knowledge as a starting point for a different but related task. Instead of training from scratch, the model begins with pre-learned features that are already useful. This is especially helpful when the new task has limited training data. The pre-trained model can be used as a fixed feature extractor or fine-tuned on the new task.

Q13. What is the difference between covariate shift and label shift?

Answer: Covariate shift occurs when the distribution of input features is different between training and test data, but the relationship between inputs and outputs stays the same. Label shift occurs when the distribution of class labels changes between training and test data. Both are types of dataset shift that can reduce model performance at deployment time and require techniques like importance weighting or domain adaptation to handle.

Q14. What is the difference between collaborative filtering and content-based filtering?

Answer: Collaborative filtering makes recommendations based on the behavior of similar users or items, using past interaction data like ratings or clicks, without needing to know what the items actually are. Content-based filtering recommends items similar to what a user liked before, based on item features like genre or description. Collaborative filtering struggles with new users or items that have no history, while content-based filtering is limited by the quality of item features.

Q15. What is the difference between global and local model interpretability?

Answer: Global interpretability explains the overall behavior of a model across all predictions, such as which features are generally most important. Local interpretability explains why the model made a specific prediction for one particular input. Methods like overall feature importance provide global insight, while techniques like SHAP and LIME explain individual predictions. Both types of interpretability are important in real-world applications where understanding model decisions builds trust.

Q16. What is feature importance and what are the two common ways to measure it?

Answer: Feature importance measures how much each input feature contributes to the model's predictions. Impurity-based importance, used in tree models, measures how much each feature reduces impurity across all splits, but it can be biased toward features with many unique values. Permutation importance measures how much model performance drops when a feature's values are randomly shuffled, breaking its relationship with the target, and is more reliable but computationally more expensive.

Q17. What is time series forecasting and what makes it different from standard regression?

Answer: Time series forecasting predicts future values based on past observations collected in time order. Unlike standard regression where data points are assumed to be independent, time series data has temporal dependencies such as trends, seasonality, and autocorrelation. These properties require careful handling, including time-based train-test splits to avoid data leakage, lag features to capture past values, and sometimes specialized models that account for time structure.

Q18. What is the difference between generative and discriminative models?

Answer: Discriminative models learn the boundary between classes by modeling the probability of the label given the input, and focus only on what is needed for classification. Generative models learn how the data itself is distributed by modeling the joint probability of inputs and labels, and can generate new data samples. Discriminative models like logistic regression usually perform better for classification, while generative models like Naive Bayes can work better with very limited data.

Q19. What is the difference between model selection and model assessment?

Answer: Model selection is the process of choosing the best algorithm or set of hyperparameters for a problem, typically done using a validation set. Model assessment is the process of estimating how well the final selected model will perform on completely new unseen data, done using a held-out test set. Mixing these two steps by using the test set for selection leads to overly optimistic performance estimates and should always be avoided.

Q20. What is nested cross-validation and why is it needed?

Answer: Nested cross-validation uses two loops of cross-validation, an outer loop for model assessment and an inner loop for model selection and hyperparameter tuning. It is needed because using the same data for both selecting a model and evaluating it produces an overly optimistic estimate of performance. The outer loop provides an unbiased estimate of generalization error while the inner loop handles all tuning decisions, keeping evaluation completely separate from selection.

Q21. What is the difference between hard and soft margin in SVM?

Answer: Hard margin SVM requires that all data points be perfectly classified with no misclassifications, which works only when data is linearly separable. Soft margin SVM introduces a slack variable that allows some misclassifications, controlled by a regularization parameter C. A smaller C allows more misclassifications for a wider margin, while a larger C tolerates fewer violations for a narrower margin.

Q22. What is the difference between generalization error, training error, and empirical risk?

Answer: Training error is the average loss measured on the training data. Empirical risk is a formal term for training error used in statistical learning theory. Generalization error is the expected loss on new unseen data drawn from the true distribution. A good model minimizes generalization error, and the gap between training and generalization error reflects how well the model generalizes beyond the training set.

Q23. What is a kernel function and what properties must it satisfy?

Answer: A kernel function measures similarity between two data points and implicitly maps them to a higher-dimensional feature space without computing the transformation explicitly. To be a valid kernel, it must satisfy Mercer's conditions, meaning the kernel matrix must be symmetric and positive semi-definite. Common kernels include the linear, polynomial, radial basis function (RBF), and sigmoid kernels.

Q24. What is the difference between bagging, boosting, and stacking in ensemble learning?

Answer: Bagging trains multiple models independently on random subsets of data and aggregates predictions to reduce variance. Boosting trains models sequentially where each corrects errors of the previous one to reduce bias. Stacking trains a meta-learner on the predictions of multiple base models to combine their strengths. Stacking is more flexible but complex, while bagging and boosting have more specific use cases.

Q25. What is LightGBM and how does it differ from XGBoost?

Answer: LightGBM is a gradient boosting framework by Microsoft that grows trees leaf-wise instead of level-wise, choosing the leaf with the highest gain to split at each step. This makes it faster and more memory-efficient than XGBoost on large datasets. It also uses histogram-based split finding and gradient-based one-side sampling (GOSS) to further accelerate training, making it particularly effective for high-dimensional and large-scale data.

Q26. What is the difference between collaborative filtering and content-based filtering?

Answer: Collaborative filtering makes recommendations based on the preferences of similar users or items, without requiring knowledge of item content, using user-item interaction history. Content-based filtering recommends items similar to those a user liked before, based on item features. Collaborative filtering suffers from the cold start problem for new users or items, while content-based filtering is limited by the quality and availability of item features.

Q27. What is the difference between online learning and batch learning?

Answer: Batch learning trains the model on the entire dataset at once and then deploys it, making it static until retrained. Online learning updates the model incrementally as each new data point or small batch arrives, allowing it to adapt continuously to new data. Online learning is useful in non-stationary environments where data distributions change over time, such as financial forecasting or real-time recommendation systems.

Q28. What is self-supervised learning and how does it differ from supervised learning?

Answer: Self-supervised learning is a form of unsupervised learning where the model generates its own supervision signals from the input data by solving pretext tasks, such as predicting masked parts of the input or the next token in a sequence. Unlike supervised learning, it does not require human-annotated labels. The learned representations can then be fine-tuned on downstream tasks with small amounts of labeled data.

Q29. What is importance weighting and how is it used to handle covariate shift?

Answer: Importance weighting corrects for covariate shift by reweighting each training example by the ratio of its probability under the test distribution to its probability under the training distribution. This ratio is called the importance weight. It makes the weighted training distribution resemble the test distribution, allowing models trained on weighted data to perform well on the test distribution. Kernel Mean Matching and density ratio estimation methods are commonly used to estimate these weights.

Q35. What is the difference between model selection, model assessment, and the double cross-validation procedure?

Answer: Model selection is the process of choosing the best algorithm or hyperparameter configuration, typically done using a validation set or inner cross-validation loop. Model assessment is the process of estimating the generalization error of the final selected model on completely unseen data using a test set or outer loop. Double (nested) cross-validation uses an outer loop for assessment and an inner loop for selection, providing an unbiased estimate of generalization performance without contaminating the test set with hyperparameter decisions.